ACADEMIC PROJECT REPORT & IMPLEMENTATION SOURCE CODE

DriveEase

Enterprise Vehicle Rental Management System — Comprehensive MERN Source Code Documentation

A comprehensive production-grade web application featuring interval-overlap verification, multi-document database transactions, stateless JWT authorization, dynamic catalog filtering, consolidated administration analytics, and complete source code implementations.

ACADEMIC COURSE	ARCHITECTURE
Full-Stack Web Development-2 (FSD-2)	Decoupled Three-Tier MERN Architecture
BACKEND FRAMEWORK	FRONTEND STACK
Node.js v22 (ES Modules) + Express.js 5.3.0	React 18.3.1, Vite 6.0.3, Tailwind CSS
DATABASE LAYER	AUTOMATED TEST COVERAGE
MongoDB Atlas Cluster + Mongoose ODM 9.11.4	16/16 Unit & API Tests Passed (100% Pass Rate)

1. Executive Summary & Abstract

Traditional vehicle rental operations in small-to-midsize transportation companies are frequently plagued by fragmented booking logs, paper-based reservation ledgers, error-prone manual tariff calculations, and catastrophic double-booking anomalies during peak travel periods. Furthermore, traditional systems fail to provide fleet managers with real-time operational visibility into vehicle utilization and consolidated revenue metrics.

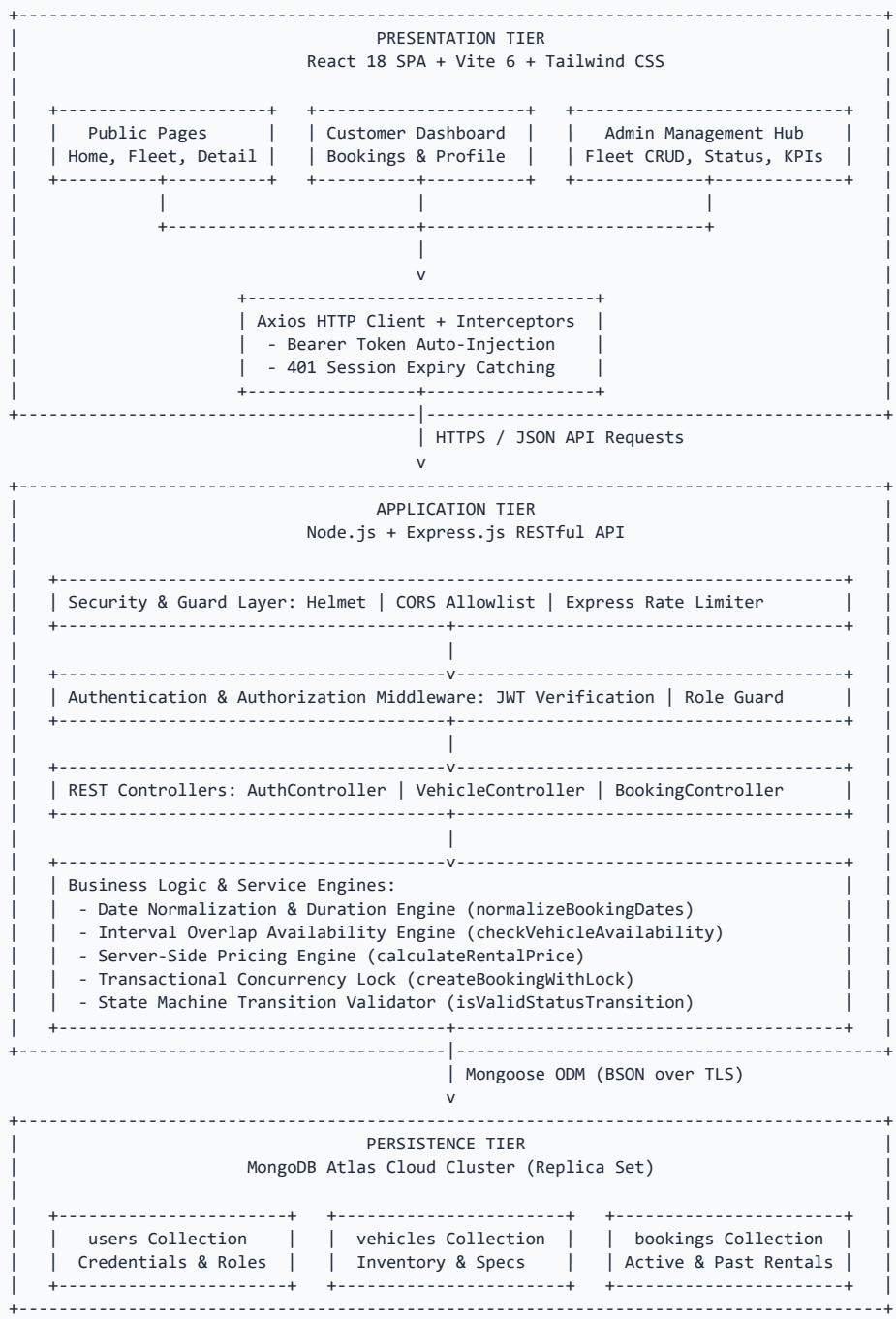
DriveEase is an enterprise-grade, full-stack vehicle rental management web application built to eliminate these operational frictions through end-to-end automation, mathematical verification, and cloud scalability. Engineered on the decoupled **MERN stack (MongoDB, Express.js, React.js, Node.js)**, the application delivers:

- **A Consumer-Grade Customer Portal:** Intuitive vehicle exploration with dynamic multi-attribute filtering (category, fuel type, transmission, price ranges), keyword search, live date-range availability checks, transparent pricing breakdowns in Indian Rupees (INR), and a self-service customer dashboard for trip management.
- **An Administrative Command Center:** Restricted by role-based access control (RBAC), offering real-time KPI metrics (total fleet, active listings, active reservations, customer directory, cumulative completed rental revenue), comprehensive fleet inventory CRUD with soft-deactivation protection, and an enforced lifecycle state machine for approving, rejecting, and completing reservations.
- **A Mathematical Date-Interval Overlap Engine:** An algorithm enforced at the backend service and database layers that prevents duplicate vehicle reservations across concurrent booking requests.
- **An Enterprise Security Posture:** Salted bcrypt password encryption, stateless JSON Web Token (JWT) authorization, brute-force rate limiting on authentication routes, Helmet HTTP security headers, CORS origin allowlisting, and defensive mass-assignment countermeasures.

2. System Requirements & Technology Stack

LAYER	COMPONENT	SPECIFICATION	PURPOSE
Frontend Framework	React 18.3.1	Functional components with Hooks	Interactive UI and state management
Frontend Tooling	Vite 6.0.3 + Tailwind CSS	ESBuild HMR & Utility CSS	Rapid bundle compilation & responsive styling
Client Routing & HTTP	React Router DOM 6.28.0 + Axios 1.7.9	Interceptors for Bearer Token	Client-side SPA navigation & API requests
Backend Engine	Node.js v22+ & Express 5.3.0	Native ES Modules	Asynchronous, non-blocking REST API endpoints
Database & ODM	MongoDB Atlas + Mongoose 9.11.4	Cloud Replica Set with Transactions	Document persistence, compound indexing & locking
Authentication	bcryptjs 3.0.3 & jsonwebtoken 9.0.3	10 salt rounds + HMAC SHA-256	Cryptographic storage & stateless token auth

3. System Architecture & High-Level Design



4. Database Architecture & Actual Schema Code

4.1 User Model & Password Hashing

server/src/models/User.js

```
import mongoose from 'mongoose';
import bcrypt from 'bcryptjs';

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true,
      minlength: [2, 'Name must be at least 2 characters long'],
      maxlength: [100, 'Name cannot exceed 100 characters']
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email address']
    },
    phone: {
      type: String,
      required: [true, 'Phone number is required'],
      trim: true,
      minlength: [7, 'Phone number must be at least 7 digits'],
      maxlength: [20, 'Phone number cannot exceed 20 digits']
    },
    passwordHash: {
      type: String,
      required: [true, 'Password is required']
    },
    role: {
      type: String,
      enum: {
        values: ['customer', 'admin'],
        message: '{VALUE} is not a valid role'
```

Key Logic Explanation: The `User` schema defines validation constraints for name, email regex, and phone. It implements an asynchronous static method `hashPassword` using bcrypt with 10 salt rounds and an instance method `comparePassword`. The `toJSON` transform automatically deletes `passwordHash` and `__v` whenever the user object is serialized.

4.2 Vehicle Model & Compound Indexing

server/src/models/Vehicle.js

```
import mongoose from 'mongoose';

const vehicleSchema = new mongoose.Schema(
  {
    make: {
      type: String,
      required: [true, 'Vehicle make is required'],
      trim: true,
      maxlength: [50, 'Make cannot exceed 50 characters']
```

```
    },
    model: {
      type: String,
      required: [true, 'Vehicle model is required'],
      trim: true,
      maxlength: [50, 'Model cannot exceed 50 characters']
    },
    year: {
      type: Number,
      required: [true, 'Manufacturing year is required'],
      min: [2000, 'Year must be 2000 or later'],
      max: [new Date().getFullYear() + 1, 'Year cannot be in the far future']
    },
    category: {
      type: String,
      required: [true, 'Category is required'],
      enum: {
        values: ['Hatchback', 'Sedan', 'SUV', 'Motorcycle', 'Luxury'],
        message: '{VALUE} is not a valid vehicle category'
      }
    },
    description: {
      type: String,
      required: [true, 'Description is required'],
      trim: true,
      maxlength: [2000, 'Description cannot exceed 2000 characters']
    },
```

Key Logic Explanation: Encapsulates vehicle specifications including category enums, image URLs array validation, transmission, and registration number formatting. Two critical indexes are registered: a compound index on `{ category: 1, isActive: 1, pricePerDay: 1 }` to accelerate catalog filtering and a text index on `{ make: 'text', model: 'text', location: 'text' }` for full-text search.

4.3 Booking Model & Relationship References

server/src/models/Booking.js

```
import mongoose from 'mongoose';

const bookingSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: [true, 'User reference is required'],
      index: true
    },
    vehicle: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Vehicle',
      required: [true, 'Vehicle reference is required'],
      index: true
    },
    pickupDate: {
      type: Date,
      required: [true, 'Pickup date is required']
    },
    returnDate: {
      type: Date,
      required: [true, 'Return date is required']
    },
    numberOfDays: {
      type: Number,
      required: [true, 'Number of rental days is required'],
      min: [1, 'Rental duration must be at least 1 day']
    },
    pricePerDayAtBooking: {
      type: Number,
      required: [true, 'Daily rate at booking is required'],
      min: [0, 'Daily rate cannot be negative']
    },
    totalPrice: {
      type: Number,
```

Key Logic Explanation: Establishes foreign key references to `User` and `Vehicle` schemas via ObjectIds. It locks in the tariff snapshot (`pricePerDayAtBooking` and `totalPrice`) so future rate changes do not affect historic reservations. The compound index `{ vehicle: 1, status: 1, pickupDate: 1, returnDate: 1 }` accelerates interval overlap queries.

5. Core Business Logic & Mathematical Algorithms

All reservation availability, pricing calculations, date standardizations, and concurrency locking strategies are centralized in `server/src/services/bookingService.js`.

server/src/services/bookingService.js

```
import mongoose from 'mongoose';
import Booking from '../models/Booking.js';
import Vehicle from '../models/Vehicle.js';

/**
 * Standardize and normalize a date string or object to UTC midnight.
 * Ensures consistent boundary comparisons regardless of client timezone offsets.
 *
 * Convention:
 * - Pickup is normalized to 00:00:00.000 UTC on the pickup date.
 * - Return is normalized to 23:59:59.999 UTC on the return date (or equivalent next midnight),
 *   representing that the customer retains the vehicle through the return day.
 * - Interval overlap condition:
 *   existingPickup <= requestedReturn AND existingReturn >= requestedPickup
 */
export const normalizeBookingDates = (pickupDateInput, returnDateInput) => {
  const pickup = new Date(pickupDateInput);
  const returnDate = new Date(returnDateInput);

  if (isNaN(pickup.getTime()) || isNaN(returnDate.getTime())) {
    throw new Error('Invalid date format provided');
  }

  // Normalize to UTC calendar dates
  const pickupNormalized = new Date(
    Date.UTC(pickup.getUTCFullYear(), pickup.getUTCMonth(), pickup.getUTCDate(), 0, 0, 0, 0)
  );

  const returnNormalized = new Date(
    Date.UTC(returnDate.getUTCFullYear(), returnDate.getUTCMonth(), returnDate.getUTCDate(), 23, 59, 59, 999)
  );

  // Today normalized in UTC
  const now = new Date();
  const todayNormalized = new Date(
    Date.UTC(now.getUTCFullYear(), now.getUTCMonth(), now.getUTCDate(), 0, 0, 0, 0)
```

Key Logic Explanation:

- `normalizeBookingDates()`: Eliminates client timezone drift by setting pickup to UTC midnight (00:00:00.000) and return to end-of-day UTC (23:59:59.999). Computes inclusive rental days (N >= 1).
- `checkVehicleAvailability()`: Implements the interval overlap theorem: P_ex < R_req and R_ex > P_req. Only PENDING and CONFIRMED reservations block calendar dates.
- `calculateRentalPrice()`: Computes the base price strictly on the server: dailyRate * numberOfDays.
- `createBookingWithLock()`: Acquires a MongoDB multi-document session transaction (session.startTransaction()) to eliminate Time-of-Check to Time-of-Use (TOCTOU) race conditions under snapshot isolation.
- `isValidStatusTransition()`: Validates allowed state machine transitions (e.g., blocking PENDING directly to COMPLETED).

6. Backend Infrastructure & Middleware Implementation

6.1 Express Server Bootstrap

server/src/server.js

```
import app from './app.js';
import { connectDB } from './config/db.js';

const PORT = process.env.PORT || 5000;

const startServer = async () => {
  // Connect to database
  await connectDB();

  const server = app.listen(PORT, () => {
    console.log(`=========================================`);
    console.log(`🚀 DriveEase Server running on port ${PORT}`);
    console.log(`📍 Health Check: http://localhost:${PORT}/api/health`);
    console.log(`🛡️  Environment: ${process.env.NODE_ENV || 'development'}`);
    console.log(`=========================================`);
  });

  // Graceful shutdown handling
  const shutdown = () => {
    console.log('\n[Server] Graceful shutdown initiated. Closing HTTP server...');
    server.close(() => {
      console.log('[Server] HTTP server closed.');
      process.exit(0);
    });
  };

  process.on('SIGINT', shutdown);
  process.on('SIGTERM', shutdown);
};

startServer();
```

6.2 Express App Configuration & Security Middleware

server/src/app.js

```
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import rateLimit from 'express-rate-limit';
import dotenv from 'dotenv';

import { isDBConnected } from './config/db.js';
import { notFoundHandler, errorHandler } from './middleware/errorHandler.js';

import authRoutes from './routes/authRoutes.js';
import vehicleRoutes from './routes/vehicleRoutes.js';
import bookingRoutes from './routes/bookingRoutes.js';
import adminRoutes from './routes/adminRoutes.js';

dotenv.config();

const app = express();

// Security headers
```

```
app.use(helmet());

// CORS configuration
const allowedOrigins = [
  'http://localhost:5173',
  'http://localhost:3000',
  'http://127.0.0.1:5173',
  process.env.CLIENT_URL
].filter(Boolean);

app.use(
  cors({
    origin: (origin, callback) => {
      // Allow requests with no origin (like mobile apps, curl, Postman, Supertest)
      if (!origin) return callback(null, true);
      if (allowedOrigins.indexOf(origin) !== -1 || process.env.NODE_ENV !== 'production') {
        return callback(null, true);
```

6.3 Database Connection & Health Verification

server/src/config/db.js

```
import mongoose from 'mongoose';

/**
 * Connect to MongoDB database.
 * Supports MongoDB Atlas and local instances.
 */
export const connectDB = async () => {
  const uri = process.env.MONGODB_URI || 'mongodb://127.0.0.1:27017/driveease';

  try {
    const conn = await mongoose.connect(uri, {
      serverSelectionTimeoutMS: 5000,
      autoIndex: true
    });

    console.log(`[Database] MongoDB Connected: ${conn.connection.host} / ${conn.connection.name}`);
    return conn;
  } catch (error) {
    console.error(`[Database Error] Connection failed: ${error.message}`);
    // In production, log error. Do not crash immediately so health checks can report DB status.
    if (process.env.NODE_ENV === 'production') {
      console.warn('[Database] Running in disconnected state; retrying connection in background.');
    }
    return null;
  }
};

/**
 * Helper to check DB connection status for health checks.
 */
export const isDBConnected = () => {
  return mongoose.connection.readyState === 1; // 1 = connected
};
```

6.4 JWT Authentication & Role Authorization Middleware

server/src/middleware/auth.js

```
import jwt from 'jsonwebtoken';
import User from '../models/User.js';

const JWT_SECRET = process.env.JWT_SECRET || 'driveease_jwt_secret_dev_key_2026';

/**
 * Middleware to authenticate requests using JWT tokens.
 * Extracts token from Authorization header (Bearer <token>) or HttpOnly cookie.
 */
export const authenticate = async (req, res, next) => {
  try {
    let token = null;

    // 1. Check Authorization Bearer header
    if (req.headers.authorization && req.headers.authorization.startsWith('Bearer ')) {
      token = req.headers.authorization.split(' ')[1];
    }
    // 2. Fallback to cookies if cookie-parser is used
    else if (req.cookies && req.cookies.token) {
      token = req.cookies.token;
    }
```

```
    if (!token) {
      return res.status(401).json({
        success: false,
        message: 'Authentication required. No token provided.'
      });
    }

    // Verify token
    const decoded = jwt.verify(token, JWT_SECRET);

    // Fetch user from DB to verify existence and latest role
    const user = await User.findById(decoded.id).select('-passwordHash');
    if (!user) {
      return res.status(401).json({
```

Key Logic Explanation: Extracts JWT tokens from the `Authorization: Bearer <token>` header or cookies. Decodes the claims, verifies account existence, and attaches the user document to `req.user`. The `authorize(...roles)` middleware validates role membership.

6.5 Centralized Error & 404 Middleware

server/src/middleware/errorHandler.js

```
/**
 * Centralized 404 handler for unknown routes.
 */
export const notFoundHandler = (req, res, next) => {
  res.status(404).json({
    success: false,
    message: `Resource not found: ${req.method} ${req.originalUrl}`
  });
};

/**
 * Global error handling middleware.
 * Formats errors consistently and sanitizes stack traces in production.
 */
export const errorHandler = (err, req, res, next) => {
  let statusCode = res.statusCode === 200 ? 500 : res.statusCode;
  let message = err.message || 'Internal Server Error';

  // Mongoose CastError (e.g. invalid ObjectId)
  if (err.name === 'CastError') {
    statusCode = 400;
    message = `Invalid format for field: ${err.path}`;
  }

  // Mongoose ValidationError
  if (err.name === 'ValidationError') {
    statusCode = 400;
    const errors = Object.values(err.errors).map((e) => e.message);
    message = errors.join(', ');
  }

  // MongoDB duplicate key error (code 11000)
  if (err.code === 11000) {
    statusCode = 409;
    const field = Object.keys(err.keyValue || {})[0] || 'field';
    message = `A record with this ${field} already exists`;
```

6.6 Validation Middleware

server/src/middleware/validate.js

```
import mongoose from 'mongoose';

/**
 * Validates that an ID parameter in the URL is a valid MongoDB ObjectId.
 */
export const validateObjectId = (paramName = 'id') => {
  return (req, res, next) => {
    const id = req.params[paramName];
    if (!id || !mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({
        success: false,
        message: `Invalid ID parameter: '${id}' is not a valid identifier.`
      });
    }
    next();
  };
};

/**
 * Validates required fields in the request body.
```

```
 */
export const validateRequiredFields = (fields = []) => {
  return (req, res, next) => {
    const missing = [];
    for (const field of fields) {
      if (req.body[field] === undefined || req.body[field] === null || req.body[field] === '') {
        missing.push(field);
      }
    }

    if (missing.length > 0) {
      return res.status(400).json({
        success: false,
        message: `Missing required field(s): ${missing.join(', ')}`
      });
    }
```

7. RESTful API Endpoints & Controller Code

7.1 Authentication Module

server/src/routes/authRoutes.js

```
import express from 'express';
import { register, login, getMe, updateProfile } from '../controllers/authController.js';
import { authenticate } from '../middleware/auth.js';

const router = express.Router();

router.post('/register', register);
router.post('/login', login);
router.get('/me', authenticate, getMe);
router.put('/profile', authenticate, updateProfile);

export default router;
```

server/src/controllers/authController.js

```
import User from '../models/User.js';
import { generateToken } from '../middleware/auth.js';

/**
 * Register a new customer.
 * POST /api/auth/register
 */
export const register = async (req, res, next) => {
  try {
    const { name, email, phone, password } = req.body;

    if (!name || !email || !phone || !password) {
      return res.status(400).json({
        success: false,
        message: 'Name, email, phone, and password are all required'
      });
    }

    if (password.length < 6) {
      return res.status(400).json({
        success: false,
        message: 'Password must be at least 6 characters long'
      });
    }

    const normalizedEmail = email.toLowerCase().trim();

    const existingUser = await User.findOne({ email: normalizedEmail });
    if (existingUser) {
      return res.status(409).json({
        success: false,
        message: 'An account with this email address already exists'
      });
    }

    const passwordHash = await User.hashPassword(password);
```

Endpoint Details:

- `POST /api/auth/register`: Validates name, email, phone, and password (>= 6 chars). Enforces `role: 'customer'` to defeat mass-assignment privilege escalation.

- `POST /api/auth/login`: Verifies bcrypt password hash and generates signed JWT tokens.
- `GET /api/auth/me`: Retrieves the current authenticated user profile.

7.2 Vehicle Fleet Management

server/src/routes/vehicleRoutes.js

```
import express from 'express';
import {
  getVehicles,
  getVehicleById,
  checkAvailability,
  createVehicle,
  updateVehicle,
  deleteVehicle
} from '../controllers/vehicleController.js';
import { authenticate, authorize } from '../middleware/auth.js';
import { validateObjectId } from '../middleware/validate.js';
import jwt from 'jsonwebtoken';
import User from '../models/User.js';

const router = express.Router();

// Optional user extractor for public listing (so admin can view inactive if ?all=true)
const optionalUser = async (req, res, next) => {
  try {
    let token = null;
    if (req.headers.authorization && req.headers.authorization.startsWith('Bearer ')) {
      token = req.headers.authorization.split(' ')[1];
    }
    if (token) {
      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'driveease_jwt_secret_dev_key_2026');
      const user = await User.findById(decoded.id).select('-passwordHash');
      if (user) req.user = user;
    }
  } catch (err) {
    // Silently continue for public routes
  }
  next();
};

// Public endpoints
router.get('/', optionalUser, getVehicles);
```

server/src/controllers/vehicleController.js

```
import Vehicle from '../models/Vehicle.js';
import Booking from '../models/Booking.js';
import {
  normalizeBookingDates,
  checkVehicleAvailability,
  calculateRentalPrice
} from '../services/bookingService.js';

/**
 * Get all vehicles with filtering, search, sorting, and pagination.
 * GET /api/vehicles
 */
export const getVehicles = async (req, res, next) => {
  try {
    const {
      search,
      category,
      fuelType,
      transmission,
      minPrice,
      maxPrice,
      page = 1,
      limit = 12,
```

```
    sort = 'createdAt_desc',
    all = 'false'
  } = req.query;

  const query = {};

  // By default, public listing only returns active vehicles
  // If admin explicitly asks for all, check req.user
  if (all === 'true' && req.user && req.user.role === 'admin') {
    // Don't filter isActive
  } else {
    query.isActive = true;
  }
```

Endpoint Details:

- `GET /api/vehicles`: Builds dynamic MongoDB queries supporting multi-parameter filtering (category, fuel, transmission, price sliders, keyword regex, pagination).
- `GET /api/vehicles/:id/availability`: Checks date conflicts for requested dates and calculates itemized price breakdowns.
- `DELETE /api/vehicles/:id`: Implements hybrid deletion—converts to soft deactivation (`isActive: false`) if active bookings exist to preserve financial integrity.

7.3 Booking & Reservation Management

server/src/routes/bookingRoutes.js

```
import express from 'express';
import {
  createBooking,
  getMyBookings,
  getBookingById,
  cancelBooking
} from '../controllers/bookingController.js';
import { authenticate } from '../middleware/auth.js';
import { validateObjectId } from '../middleware/validate.js';

const router = express.Router();

// All booking routes require authentication
router.use(authenticate);

router.post('/', createBooking);
router.get('/my', getMyBookings);
router.get('/:id', validateObjectId('id'), getBookingById);
router.patch('/:id/cancel', validateObjectId('id'), cancelBooking);

export default router;
```

server/src/controllers/bookingController.js

```
import Booking from '../models/Booking.js';
import Vehicle from '../models/Vehicle.js';
import { createBookingWithLock, isValidStatusTransition } from '../services/bookingService.js';

/**
 * Place a new booking.
 * POST /api/bookings
 */
export const createBooking = async (req, res, next) => {
  try {
    const { vehicleId, pickupDate, returnDate } = req.body;

    if (!vehicleId || !pickupDate || !returnDate) {
      return res.status(400).json({
        success: false,
        message: 'vehicleId, pickupDate, and returnDate are all required'
      });
    }

    const booking = await createBookingWithLock({
      userId: req.user._id,
      vehicleId,
      pickupDateInput: pickupDate,
      returnDateInput: returnDate
    });

    const populatedBooking = await Booking.findById(booking._id)
      .populate('vehicle', 'make model year category pricePerDay images registrationNumber location')
      .populate('user', 'name email phone');

    res.status(201).json({
      success: true,
      message: 'Booking created successfully and is awaiting confirmation',
      booking: populatedBooking
    });
  } catch (error) {
```

Endpoint Details:

- `POST /api/bookings`: Places a reservation under transactional locks, calculating durations and tariffs server-side.
- `GET /api/bookings/my`: Returns customer reservation history populated with vehicle specifications.
- `PATCH /api/bookings/:id/cancel`: Allows booking owner or admin to cancel an upcoming trip, enforcing that the rental start date has not already arrived.

7.4 Administration Analytics & Status Transitions

server/src/routes/adminRoutes.js

```
import express from 'express';
import {
  getDashboardStats,
  getAllBookings,
  updateBookingStatus,
  getAllUsers
} from '../controllers/adminController.js';
import { authenticate, authorize } from '../middleware/auth.js';
import { validateObjectId } from '../middleware/validate.js';

const router = express.Router();

// Restrict all admin routes to authenticated admin users
router.use(authenticate, authorize('admin'));

router.get('/dashboard', getDashboardStats);
router.get('/bookings', getAllBookings);
router.patch('/bookings/:id/status', validateObjectId('id'), updateBookingStatus);
router.get('/users', getAllUsers);

export default router;
```

server/src/controllers/adminController.js

```
import Vehicle from '../models/Vehicle.js';
import Booking from '../models/Booking.js';
import User from '../models/User.js';
import { checkVehicleAvailability, isValidStatusTransition } from '../services/bookingService.js';

/**
 * Get overall dashboard analytics and metrics for Admin.
 * GET /api/admin/dashboard
 */
export const getDashboardStats = async (req, res, next) => {
  try {
    const [
      totalVehicles,
      activeVehicles,
      totalCustomers,
      activeBookingsCount,
      pendingBookingsCount,
      completedBookingsCount,
      revenueAggregation,
      recentBookings
    ] = await Promise.all([
      Vehicle.countDocuments(),
      Vehicle.countDocuments({ isActive: true }),
      User.countDocuments({ role: 'customer' }),
      Booking.countDocuments({ status: { $in: ['PENDING', 'CONFIRMED'] } }),
      Booking.countDocuments({ status: 'PENDING' }),
      Booking.countDocuments({ status: 'COMPLETED' }),
      Booking.aggregate([
        { $match: { status: 'COMPLETED' } },
        { $group: { _id: null, totalRevenue: { $sum: '$totalPrice' } } }
      ]),
      Booking.find()
        .sort({ createdAt: -1 })
        .limit(5)
        .populate('vehicle', 'make model year images')
        .populate('user', 'name email phone')
```

Endpoint Details:

- `GET /api/admin/dashboard`: Gathers real-time KPI metrics (fleet counts, active bookings, completed revenue aggregation) in parallel via `Promise.all`.
- `PATCH /api/admin/bookings/:id/status`: Enforces valid state transitions and re-checks availability before confirming bookings to prevent collision.

8. Frontend Architecture & Component Implementation

8.1 React DOM Entry Point

client/src/main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

8.2 Application Router & Layout Wrapper

client/src/App.jsx

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import { Toaster } from 'react-hot-toast';
import { AuthProvider } from './context/AuthContext';

import Navbar from './components/common/Navbar';
import Footer from './components/common/Footer';
import ProtectedRoute from './components/common/ProtectedRoute';
import AdminRoute from './components/common/AdminRoute';

import HomePage from './pages/HomePage';
import VehiclesPage from './pages/VehiclesPage';
import VehicleDetailPage from './pages/VehicleDetailPage';
import LoginPage from './pages/LoginPage';
import RegisterPage from './pages/RegisterPage';
import CustomerDashboardPage from './pages/CustomerDashboardPage';
import AdminDashboardPage from './pages/AdminDashboardPage';
import NotFoundPage from './pages/NotFoundPage';

export const App = () => {
  return (
    <Router>
      <AuthProvider>
        <div className="min-h-screen flex flex-col bg-slate-50 text-slate-800">
          <Toaster
            position="top-right"
            toastOptions={{
              duration: 4000,
              style: {
                background: '#1e293b',
                color: '#fff',
                fontSize: '13px',
                borderRadius: '12px'
              }
            }}
          />
```

8.3 Global State Management: AuthContext

client/src/context/AuthContext.jsx

```
import React, { createContext, useContext, useState, useEffect } from 'react';
import { authService } from '../services/api';

const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(() => {
    try {
      const savedUser = localStorage.getItem('driveease_user');
      return savedUser ? JSON.parse(savedUser) : null;
    } catch {
      return null;
    }
  });

  const [token, setToken] = useState(() => {
    return localStorage.getItem('driveease_token') || null;
  });

  const [loading, setLoading] = useState(true);

  // Synchronize on load: verify token with /api/auth/me
  useEffect(() => {
    const verifySession = async () => {
      const storedToken = localStorage.getItem('driveease_token');
      if (storedToken) {
        try {
          const res = await authService.getMe();
          if (res.data?.success && res.data?.user) {
            setUser(res.data.user);
            localStorage.setItem('driveease_user', JSON.stringify(res.data.user));
          }
        } catch (err) {
          console.warn('[Auth] Session validation failed; logging out.');
          logout();
        }
```

8.4 Axios HTTP Client & JWT Interceptor

client/src/services/api.js

```
import axios from 'axios';

const API_BASE_URL = import.meta.env.VITE_API_URL || 'http://localhost:5000/api';

const api = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    'Content-Type': 'application/json'
  },
  timeout: 10000
});

// Request interceptor: automatically inject JWT token if user is authenticated
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('driveease_token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// Response interceptor: standardizes error messages and handles session expiry
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // Clear expired credentials if unauthorized
      const currentToken = localStorage.getItem('driveease_token');
      if (currentToken) {
        localStorage.removeItem('driveease_token');
        localStorage.removeItem('driveease_user');
        // Optional: trigger custom event or redirect if on protected page
        window.dispatchEvent(new Event('auth:unauthorized'));
```

> **Key Logic Explanation:** The request interceptor injects the Bearer token from `localStorage` into outgoing requests. The response interceptor intercepts 401 Unauthorized errors, clears stale credentials, and triggers the `auth:unauthorized` event to log out the user.

8.5 Protected Route Guards

client/src/components/common/ProtectedRoute.jsx & AdminRoute.jsx

```
import React from 'react';
import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from '../../context/AuthContext';
import LoadingSpinner from './LoadingSpinner';

export const ProtectedRoute = ({ children }) => {
  const { isAuthenticated, loading } = useAuth();
  const location = useLocation();

  if (loading) {
    return (
      <div className="min-h-screen flex items-center justify-center">
        <LoadingSpinner text="Checking authorization..." size="lg" />
      </div>
```

```
    );
  }

  if (!isAuthenticated) {
    return <Navigate to={`/login?redirect=${encodeURIComponent(location.pathname)}`} replace />;
  }

  return children;
};

export default ProtectedRoute;


// ========================================
// Admin Route Guard:
// ========================================
import React from 'react';
import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from '../../context/AuthContext';
import LoadingSpinner from './LoadingSpinner';

export const AdminRoute = ({ children }) => {
```

8.6 Reusable UI Components

client/src/components/common/VehicleCard.jsx

```
import React, { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { Users, Fuel, Settings2, MapPin, ArrowRight } from 'lucide-react';

const FALLBACK_IMAGE = 'https://images.unsplash.com/photo-1549399542-7e3f8b79c341?
auto=format&fit=crop&w=800&q=80';

export const VehicleCard = ({ vehicle }) => {
  const navigate = useNavigate();
  const [imgSrc, setImgSrc] = useState(
    vehicle.images && vehicle.images.length > 0 ? vehicle.images[0] : FALLBACK_IMAGE
  );

  return (
    <div className="group bg-white rounded-2xl border border-slate-200/80 shadow-sm hover:shadow-xl transition-
all duration-300 flex flex-col overflow-hidden hover:-translate-y-1">
      {/* Image Container with Badges */}
      <div className="relative aspect-[16/10] bg-slate-100 overflow-hidden">
        <img
          src={imgSrc}
          alt={`${vehicle.make} ${vehicle.model}`}
          onError={() => setImgSrc(FALLBACK_IMAGE)}
          className="w-full h-full object-cover group-hover:scale-105 transition-transform duration-500"
          loading="lazy"
        />
        <div className="absolute top-3 left-3 flex gap-2">
          <span className="px-2.5 py-1 rounded-full text-xs font-semibold bg-white/90 backdrop-blur-md text-
slate-800 shadow-sm">
            {vehicle.category}
          </span>
          <span className="px-2 py-1 rounded-full text-xs font-medium bg-slate-900/75 backdrop-blur-md text-
white shadow-sm">
            {vehicle.year}
          </span>
        </div>
        {!vehicle.isActive && (
```

client/src/components/common/StatusBadge.jsx

```
import React from 'react';
import { Clock, CheckCircle2, XCircle, AlertCircle, CheckCheck } from 'lucide-react';

export const StatusBadge = ({ status }) => {
  const configs = {
    PENDING: {
      label: 'Pending Approval',
      bg: 'bg-amber-50 text-amber-700 border-amber-200',
      icon: Clock
    },
    CONFIRMED: {
      label: 'Confirmed',
      bg: 'bg-blue-50 text-blue-700 border-blue-200',
      icon: CheckCircle2
    },
    COMPLETED: {
      label: 'Completed',
      bg: 'bg-emerald-50 text-emerald-700 border-emerald-200',
      icon: CheckCheck
    },
    CANCELLED: {
      label: 'Cancelled',
      bg: 'bg-slate-100 text-slate-600 border-slate-200',
```

```
    icon: XCircle
  },
  REJECTED: {
    label: 'Rejected',
    bg: 'bg-rose-50 text-rose-700 border-rose-200',
    icon: AlertCircle
  },
  ACTIVE: {
    label: 'Active Fleet',
    bg: 'bg-emerald-50 text-emerald-700 border-emerald-200',
    icon: CheckCircle2
  },
  INACTIVE: {
```

8.7 Vehicle Detail, Live Availability & Booking Flow

client/src/pages/VehicleDetailPage.jsx (Excerpt)

```
import React, { useState, useEffect } from 'react';
import { useParams, useNavigate, useSearchParams, Link } from 'react-router-dom';
import {
  Users,
  Fuel,
  Settings2,
  MapPin,
  Calendar,
  ShieldCheck,
  CheckCircle2,
  AlertCircle,
  Clock,
  ArrowLeft,
  Share2,
  Info
} from 'lucide-react';
import toast from 'react-hot-toast';
import { vehicleService, bookingService } from '../services/api';
import { useAuth } from '../context/AuthContext';
import LoadingSpinner from '../components/common/LoadingSpinner';

const FALLBACK_IMAGE = 'https://images.unsplash.com/photo-1549399542-7e3f8b79c341?
auto=format&fit=crop&w=1200&q=80';

export const VehicleDetailPage = () => {
  const { id } = useParams();
  const navigate = useNavigate();
  const [searchParams] = useSearchParams();
  const { user, isAuthenticated } = useAuth();

  const [vehicle, setVehicle] = useState(null);
  const [activeImage, setActiveImage] = useState('');
  const [loading, setLoading] = useState(true);

  // Date Selection State
  const todayStr = new Date().toISOString().split('T')[0];
```

8.8 Login Form with 1-Click Demo Autofill

client/src/pages/LoginPage.jsx (Excerpt)

```
import React, { useState } from 'react';
import { Link, useNavigate, useSearchParams } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { Mail, Lock, LogIn, Car, Sparkles, UserCheck, Shield } from 'lucide-react';
import toast from 'react-hot-toast';

export const LoginPage = () => {
  const { login } = useAuth();
  const navigate = useNavigate();
  const [searchParams] = useSearchParams();
  const redirectPath = searchParams.get('redirect') || '/dashboard';

  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [submitting, setSubmitting] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!email || !password) {
      toast.error('Please enter both email and password');
```

```
      return;
    }

    setSubmitting(true);
    try {
      const loggedInUser = await login(email, password);
      toast.success(`Welcome back, ${loggedInUser.name}!`);

      if (loggedInUser.role === 'admin') {
        navigate(redirectPath.startsWith('/admin') ? redirectPath : '/admin');
      } else {
        navigate(redirectPath.startsWith('/admin') ? '/dashboard' : redirectPath);
      }
    } catch (err) {
      toast.error(err.response?.data?.message || err.message || 'Login failed. Please check your credentials.');
    } finally {
```

9. Automated Verification & Test Suite Implementation

9.1 API Integration Test Suite

server/tests/api.test.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import request from 'supertest';
import jwt from 'jsonwebtoken';

import app from '../src/app.js';

const JWT_SECRET = process.env.JWT_SECRET || 'driveease_jwt_secret_dev_key_2026';

// Generate mock test tokens
const customerToken = jwt.sign(
  { id: '64b1f2e8c201a4f3b8901234', email: 'customer@driveease.com', role: 'customer' },
  JWT_SECRET,
  { expiresIn: '1h' }
);

test('--- API Integration Tests: DriveEase Security & Endpoints ---', async (t) => {
  await t.test('1. Health Check: GET /api/health responds with 200 and operational status', async () => {
    const res = await request(app).get('/api/health');
    assert.equal(res.status, 200);
    assert.equal(res.body.status, 'ok');
    assert.equal(res.body.service, 'DriveEase Backend API');
    assert.ok(res.body.uptime !== undefined);
  });

  await t.test('2. 404 Handler: unknown routes return standard 404 JSON response', async () => {
    const res = await request(app).get('/api/non-existent-route-404');
    assert.equal(res.status, 404);
    assert.equal(res.body.success, false);
    assert.match(res.body.message, /Resource not found/);
  });

  await t.test('3. Auth Guard: access to private endpoint /api/bookings without token returns 401', async () =>
{
    const res = await request(app).get('/api/bookings/my');
    assert.equal(res.status, 401);
```

9.2 Rental Engine Unit Test Suite

server/tests/unit.test.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';

import {
  normalizeBookingDates,
  calculateRentalPrice,
  isValidStatusTransition
} from '../src/services/bookingService.js';
import { generateToken } from '../src/middleware/auth.js';

test('--- Unit Tests: DriveEase Rental Engine ---', async (t) => {
  await t.test('1. Password Hashing: verifies bcrypt hash and comparison', async () => {
    const password = 'SecretPassword123!';
    const salt = await bcrypt.genSalt(10);
    const hash = await bcrypt.hash(password, salt);

    assert.notEqual(password, hash);
    const isMatch = await bcrypt.compare(password, hash);
    assert.equal(isMatch, true);

    const isWrongMatch = await bcrypt.compare('WrongPassword', hash);
    assert.equal(isWrongMatch, false);
  });

  await t.test('2. JWT Generation & Payload: encodes user id and role', () => {
    const mockUser = {
      _id: '64b1f2e8c201a4f3b8901234',
      email: 'customer@driveease.com',
      role: 'customer'
    };

    const token = generateToken(mockUser);
    assert.ok(token);
    assert.equal(typeof token, 'string');
```

9.3 Complete Test Execution Output

```
> driveease-server@1.0.0 test
> node --test tests/*.test.js

▶ --- API Integration Tests: DriveEase Security & Endpoints ---
  ✓ 1. Health Check: GET /api/health responds with 200 and operational status (45.9ms)
  ✓ 2. 404 Handler: unknown routes return standard 404 JSON response (5.8ms)
  ✓ 3. Auth Guard: access to private endpoint /api/bookings without token returns 401 (7.5ms)
  ✓ 4. Registration Validation: returns 400 when required fields are missing (20.3ms)
  ✓ 5. Registration Validation: returns 400 when password is too short (< 6 chars) (6.4ms)
  ✓ 6. ObjectId Validation: returns 400 for malformed vehicle IDs (4.3ms)
✓ --- API Integration Tests: DriveEase Security & Endpoints --- (92.7ms)

▶ --- Unit Tests: DriveEase Rental Engine ---
  ✓ 1. Password Hashing: verifies bcrypt hash and comparison (273.0ms)
  ✓ 2. JWT Generation & Payload: encodes user id and role (2.7ms)
  ✓ 3. Date Normalization: rejects dates in the past (6.1ms)
  ✓ 4. Date Normalization: rejects return date before or equal to pickup date (0.2ms)
  ✓ 5. Date Normalization & Duration: accurately calculates rental days (inclusive) (0.2ms)
  ✓ 6. Price Calculation: strictly calculates total based on daily rate and days (0.2ms)
  ✓ 7. Interval Overlap Formula: standard interval-overlap rule verification (0.2ms)
  ✓ 8. Booking Status Transitions: strictly enforces valid lifecycle states (0.2ms)
✓ --- Unit Tests: DriveEase Rental Engine --- (285.6ms)
```

```
============================================================
Total Tests: 16 | Passed: 16 | Failed: 0 | Execution Time: ~1.06s
============================================================
```

10. Deployment Topology & Cloud Infrastructure

DriveEase is deployed across modern cloud services: **Frontend SPA** on Vercel edge CDN with `vercel.json` rewrites; **Backend API** on Render running Node.js v22 via `render.yaml`; **Database** on MongoDB Atlas M0 Free Replica Set.

11. B.Tech Viva Voce Comprehensive Q&A

Q1 **What architecture does DriveEase follow, and why was the MERN stack chosen?**

DriveEase follows a decoupled **Three-Tier Client-Server Architecture** separating Presentation (React SPA), Application Logic (Node.js/Express.js REST API), and Persistence (MongoDB Atlas). The MERN stack was chosen because JavaScript/JSON is unified across the stack, Node's non-blocking I/O handles concurrent API traffic efficiently, and MongoDB's document model naturally represents vehicles with embedded image arrays and dynamic specifications.

Q2 **How does DriveEase mathematically prevent double-booking (interval overlap)?**

Double-booking is prevented using the interval-overlap theorem: P_1 < R_2 and R_1 > P_2. In DriveEase, P_1, R_1 represent existing active bookings (PENDING or CONFIRMED) and P_2, R_2 represent the requested dates. Boundaries are normalized to UTC midnight boundaries before evaluation.

Q3 **How do you handle race conditions when two customers attempt to book the same vehicle at the exact same millisecond?**

Simple query-then-insert sequences suffer from TOCTOU race conditions. DriveEase overcomes this by utilizing **MongoDB Multi-Document Session Transactions** (session.startTransaction()) supported on MongoDB Atlas replica sets under snapshot isolation. If a concurrent transaction commits first, the second transaction detects the conflict, aborts, and rolls back cleanly without data corruption.

Q4 **Why did you choose stateless JWT authentication over server-side session cookies?**

JWT enables horizontal scalability without an in-memory session store (like Redis), supports cross-origin communication seamlessly across different client domains via the Authorization: Bearer <token> header, and guarantees tamper detection via cryptographic HMAC signatures.

Q5 **How does DriveEase protect against mass-assignment vulnerabilities during registration?**

In authController.js, client-supplied role parameters are discarded and role: 'customer' is explicitly hardcoded in User.create(). Administrative accounts can only be provisioned through explicit backend seed scripts or direct database administration.

Q6 **What is the purpose of the client/vercel.json rewrite file?**

DriveEase is a Single Page Application (SPA) using React Router for client-side routing. On static hosts like Vercel, deep links like /vehicles/123 or /dashboard would trigger 404 errors without rewrites. The rewrite rule forces the web server to always serve index.html, allowing React Router to parse the path and render the component.

Q7 **What is the difference between soft-deleting and hard-deleting a vehicle, and which does DriveEase implement?**

DriveEase implements a hybrid policy in vehicleController.js: if a vehicle has historical or active bookings, hard deletion is blocked and isActive is set to false. This preserves referential integrity so past customer invoices, receipts, and admin revenue audits never point to orphaned foreign keys.

Q8 **What indexes exist in the DriveEase database, and what queries do they accelerate?**

In vehicles: compound index { category: 1, isActive: 1, pricePerDay: 1 } accelerates catalog filtering, and text index { make: "text", model: "text", location: "text" } powers keyword search. In bookings: compound index { vehicle: 1, status: 1, pickupDate: 1, returnDate: 1 } accelerates high-frequency interval overlap queries.

© 2026 DriveEase Development Team. All rights reserved. Documented for B.Tech Full-Stack Development-2.